claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-adc254ef035cb30fc.jsonl`
- SHA-256: `9f0cf939c5d1b7f28f286a60786c3765cc911c70591b592b0e90867309efbb29`
- Source modified: `2026-06-10T16:24:15+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:22:08.042Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Key rotation overwrites the only copy of the current key before the file is rekeyed; rollback misses realistic failures
SEVERITY CLAIMED: critical
FILE: src/muneem/cli.py (line 438-446)
DESCRIPTION: `db rotate-key` writes the fresh key into the keychain FIRST (`db_key.set_key(fresh)`), destroying the only durable copy of the current key (it survives only in process memory as `current`), and only then rekeys the file. Two realistic failure modes permanently lock the ledger: (1) a crash/power loss/Ctrl-C between `set_key(fresh)` and a completed `PRAGMA rekey` leaves keychain=fresh, file=old-key, old key gone; (2) the rollback `except` only catches `dbmod.EncryptedDatabaseError`, but `rekey_database` (src/muneem/db.py:274-279) can raise `sqlcipher3.OperationalError` (e.g. 'database is locked' ? very plausible on Windows with another process/AV holding the file) or any other sqlcipher3 error from `PRAGMA rekey`, which escapes uncaught, skips the rollback, and exits with keychain=fresh / file=old. Unless the user happens to have a recovery file (which wraps the OLD key), the entire financial ledger and all same-key backups are unrecoverable. The inline comment ('Keychain-first with rollback: if the file rekey fails, the keychain still matches the file') asserts the opposite of what the code does ? keychain-first guarantees a mismatch window.
EVIDENCE QUOTED: # Keychain-first with rollback: if the file rekey fails, the keychain still matches the file.
    db_key.set_key(fresh)
    try:
        dbmod.rekey_database(db_path, current, fresh)
    except dbmod.EncryptedDatabaseError as exc:
        db_key.set_key(current) # roll back; the file is still under the old key
SUGGESTED FIX: Never have a moment where only one key exists in durable storage: write the fresh key under a secondary keychain name (e.g. 'db-key-pending') first, rekey the file, then promote it to 'db-key' and delete the pending entry; on open failure, fall back to trying the pending key. At minimum: rekey the file BEFORE touching the 'db-key' entry, broaden the except to catch all exceptions (re-raising after rollback), and stash the old key under 'db-key-prev' before overwriting.

Your job is to try to REFUTE it. Your lens: existing-mitigation: check whether something ELSE in the repo (a test, a guard, a caller-side check, a hook, config) already prevents or detects the claimed problem.
Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL

financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service. Structured output only.

2. user (2026-06-10T16:22:13.555Z)

```
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
390     ) -> None:
391         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
392         from muneem import db as dbmod
393         from muneem.db_key import get_or_create_key
394
395         db_path = db or load_config().db_path
396         if not Path(db_path).exists():
397             typer.echo(
398                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
399             )
400             return
401         if not dbmod.is_plaintext_sqlite(db_path):
402             typer.echo(
403                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
404             )
405             return
406         n = dbmod.encrypt_database(db_path, get_or_create_key())
407         typer.echo(
408             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
409             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
410         )
411
412
413     def _require_encrypted_ledger(db_path: Path) -> str:
414         """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
415         from muneem import db as dbmod
416         from muneem.db_key import peek_key
417
418         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
419             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
err=True)
420             raise typer.Exit(1)
421         key = peek_key()
422         if not key:
423             typer.echo("no encryption key in the keychain for this ledger.", err=True)
424             raise typer.Exit(1)
425         return key
```

```
426
427
428     @db_app.command("rotate-key")
429     def db_rotate_key(
430         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
431     ) -> None:
432         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
433         from muneem import db as dbmod
434         from muneem import db_key
435
436         db_path = db or load_config().db_path
437         current = _require_encrypted_ledger(db_path)
438         fresh = db_key.new_key()
439         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
the file.
440         db_key.set_key(fresh)
441         try:
442             dbmod.rekey_database(db_path, current, fresh)
443         except dbmod.EncryptedDatabaseError as exc:
444             db_key.set_key(current)  # roll back; the file is still under the old key
445             typer.echo(str(exc), err=True)
446             raise typer.Exit(1) from exc
447         typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
448
449
450     @db_app.command("backup")
451     def db_backup(
452         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
453         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
454     ) -> None:
455         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
456         from muneem import db as dbmod
457
458         db_path = db or load_config().db_path
459         key = _require_encrypted_ledger(db_path)
460         try:
461             dbmod.backup_database(db_path, dest, key)
462         except dbmod.EncryptedDatabaseError as exc:
463             typer.echo(str(exc), err=True)
464             raise typer.Exit(1) from exc
465         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
466
467
468     @db_app.command("setup-recovery")
469     def db_setup_recovery(
470         out: Path | None = typer.Option(
471             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
472         ),
473     ) -> None:
474         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
475
476         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
477         file holds only the *wrapped* key ? useless without the passphrase.
478         """
479         import getpass
480
481         from muneem import recovery
482         from muneem.db_key import peek_key
483
484         key = peek_key()
```

```
485         if not key:
486             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487             raise typer.Exit(1)
488         out_path = out or (load_config().data_dir / "muneem-recovery.json")
489         passphrase = getpass.getpass("New recovery passphrase: ")
```

3. user (2026-06-10T16:22:14.598Z)

```
230             try:
231                 src_tables = src.execute(
232                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233                 ).fetchone()[0]
234                 target = str(enc_tmp).replace("\\", "/")  # SQL string literal: forward
slashes
235                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY \"x'{key}'\"")
236                 src.execute("SELECT sqlcipher_export('encrypted')")
237                 src.execute("DETACH DATABASE encrypted")
238             finally:
239                 src.close()
240
241             # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242             # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243             # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244             with connect(str(enc_tmp), key=key) as check:
245                 integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246                 ntables = check.execute(
247                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248                 ).fetchone()[0]
249             if integrity != "ok" or ntables != src_tables:
250                 raise EncryptedDatabaseError(
251                     f"encryption migration verification failed for {path.name} "
252                     f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253                 )
254
255             backup = path.with_name(path.name + ".plaintext.bak")
256             backup.unlink(missing_ok=True)
257             path.rename(backup)
258             enc_tmp.rename(path)  # consumed on success
259             _harden_permissions(str(path))
260         finally:
261             # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262             enc_tmp.unlink(missing_ok=True)
263         return int(ntables)
264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key)  # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = \"x'{new_key}'\"")
277             conn.commit()
278         finally:
```

```
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key)  # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:
299                 target.execute(_raw_key_pragma(key))  # key the destination so the backup is
encrypted
300                 source.backup(target)  # consistent snapshot of the live DB
301             finally:
302                 target.close()
303         finally:
304             source.close()
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313         """
314         from muneem import migrations
315
316         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
```

4. user (2026-06-10T16:22:21.603Z)

```
1       """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3       muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4       The 256-bit key is generated on first use and stored in the OS keychain via
5       :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6       never on disk in plaintext, never in the repo, never logged.
7
8       It is handed to SQLCipher in **raw-key form** (``PRAGMA key = "x'<64 hex>'"``), which
uses
9       the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10      key is already full-entropy random; KDF stretching only buys security for low-entropy
human
```

```
11      passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12      """
13
14      from __future__ import annotations
15
16      import secrets as _stdlib_secrets
17
18      from muneem import secrets as _keychain
19
20      # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21      _DB_KEY_NAME = "db-key"
22
23
24      def get_or_create_key() -> str:
25          """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26          existing = _keychain.get_secret(_DB_KEY_NAME)
27          if existing:
28              return existing
29          key = _stdlib_secrets.token_bytes(32).hex()
30          _keychain.set_secret(_DB_KEY_NAME, key)
31          return key
32
33
34      def peek_key() -> str | None:
35          """Return the stored key, or ``None`` if none has been minted yet. Never creates
one."""
36          return _keychain.get_secret(_DB_KEY_NAME)
37
38
39      def new_key() -> str:
40          """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41          return _stdlib_secrets.token_bytes(32).hex()
42
43
44      def set_key(key: str) -> None:
45          """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46          _keychain.set_secret(_DB_KEY_NAME, key)
47
```

5. user (2026-06-10T16:22:22.322Z)

```
Found 7 files
src\muneem\db.py
docs\feedbackJune2026.md
docs\CURRENT_STATE.md
src\muneem\cli.py
docs\encryption-at-rest.md
src\muneem\db_key.py
tests\test_db_crypto_ops.py
```

6. user (2026-06-10T16:22:28.933Z)

```
100         particulars TEXT,
101         ref TEXT,
102         withdrawal REAL,
103         deposit REAL,
104         closing_balance REAL
105     );
106     """
107
108
109     def _raw_key_pragma(key: str) -> str:
```

```
        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
        return f"PRAGMA key = \"x'{key}'\""


    def is_plaintext_sqlite(db_path: Path | str) -> bool:
        """True iff the file exists and begins with the *unencrypted* SQLite magic header.

        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
        without needing the key.
        """
        try:
            with open(db_path, "rb") as fh:
                return fh.read(16) == _SQLITE_MAGIC
        except OSError:
            return False


    def _harden_permissions(path: str) -> None:
        """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``."""
        if path == ":memory:":
            return
        try:
            os.chmod(path, 0o600)
        except OSError:
            pass


    def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
        """Open an encrypted connection via SQLCipher, verifying the key before returning.

        A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
the
        key is wrong (or the file is not an encrypted muneem ledger), this raises.
        """
        try:
            import sqlcipher3
        except ImportError as exc:  # pragma: no cover - exercised only without the binding
            raise EncryptedDatabaseError(
                "database encryption requested but the 'sqlcipher3' package is not
installed"
            ) from exc

        conn = sqlcipher3.connect(path)
        conn.execute(_raw_key_pragma(key))  # MUST be the first statement on the connection
        try:
            conn.execute("SELECT count(*) FROM sqlite_master")  # forces key verification
        except sqlcipher3.DatabaseError as exc:
            conn.close()
            raise EncryptedDatabaseError(
                f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
                "encrypted muneem ledger"
            ) from exc
        conn.row_factory = sqlcipher3.Row
        return conn


    @contextmanager
    def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
        """Open a connection (commit on success, always close). Parent dirs created.

        With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
```

```
        SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
        ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
        :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
        """
        path = str(db_path)
        if path != ":memory:":
            Path(path).parent.mkdir(parents=True, exist_ok=True)
        if key is not None:
            conn = _open_encrypted(path, key)
            _harden_permissions(path)
        else:
            conn = sqlite3.connect(path)
            conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        try:
            yield conn
            conn.commit()
        finally:
            conn.close()


    @contextmanager
    def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
        """Open the real on-disk ledger, encrypted with the keychain-managed key.

        The production path: resolves (or mints, on first use) the 256-bit key from the OS
        keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
        file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
        "file is not a database" error.
        """
        from muneem.db_key import get_or_create_key

        path = str(db_path)
        if is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
                "to migrate it to an encrypted ledger"
            )
        with connect(path, key=get_or_create_key()) as conn:
            yield conn


    def encrypt_database(db_path: Path | str, key: str) -> int:
        """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.

        The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
        securely delete (it still holds the cleartext data). Returns the number of user
tables
        copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
        """
        import sqlcipher3

        path = Path(db_path)
        if not is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
            )
```

```
225
226         enc_tmp = path.with_name(path.name + ".enc.tmp")
227         enc_tmp.unlink(missing_ok=True)
228         try:
229             src = sqlcipher3.connect(str(path))  # open the plaintext source (no key)
```

7. user (2026-06-10T16:22:30.173Z)

```
26-    with dbmod.connect(path, key=key) as conn:
27-        conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
28-        conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(n)])
29-
30-
31:# --- rekey / rotation
------------------------------------------------------------------------
32-
33-
34:def test_rekey_changes_the_key_and_keeps_data(tmp_path):
35-    p = tmp_path / "led.db"
36-    _seed_encrypted(p, KEY_A, n=5)
37:    dbmod.rekey_database(p, KEY_A, KEY_B)
38-    with pytest.raises(dbmod.EncryptedDatabaseError):  # old key no longer opens it
39-        with dbmod.connect(p, key=KEY_A):
40-            pass
41-    with dbmod.connect(p, key=KEY_B) as conn:  # new key works, data intact
42-        assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 5
43-
44-
45:def test_rekey_with_wrong_current_key_raises(tmp_path):
46-    p = tmp_path / "led.db"
47-    _seed_encrypted(p, KEY_A)
48-    with pytest.raises(dbmod.EncryptedDatabaseError):
49:        dbmod.rekey_database(p, KEY_B, KEY_A)
50-
51-
52-# --- encrypted backup
------------------------------------------------------------------------
53-
54-
--
113-        with dbmod.connect_encrypted(p):
114-            pass
115-    assert "muneem db encrypt" in str(ei.value)  # actionable guidance, not a cryptic error
116-
117-
118:# --- CLI: db rotate-key / db backup
------------------------------------------------------------
119-
120-
121-def _new_encrypted_ledger(path):
122-    with dbmod.connect_encrypted(path) as conn:
123-        dbmod.init_db(conn)
124-        dbmod.insert_document(conn, source="file", sha256="s", status="parsed")
125-
126-
127:def test_db_rotate_key_cli(tmp_path):
128-    led = tmp_path / "ledger.db"
129-    _new_encrypted_ledger(led)
130-    old = db_key.peek_key()
131:    res = runner.invoke(app, ["db", "rotate-key", "--db", str(led)])
132-    assert res.exit_code == 0, res.output
133-    new = db_key.peek_key()
134-    assert new and new != old
135-    with dbmod.connect(led, key=new) as conn:  # opens under the new key
```

```
136-        assert dbmod.get_document_by_sha(conn, "s") is not None
--
148-    assert dest.exists() and not dbmod.is_plaintext_sqlite(dest)
149-    with dbmod.connect(dest, key=db_key.peek_key()) as conn:
150-        assert dbmod.get_document_by_sha(conn, "s") is not None
151-
152-
153:def test_db_rotate_key_refuses_when_no_encrypted_ledger(tmp_path):
154:    res = runner.invoke(app, ["db", "rotate-key", "--db", str(tmp_path / "nope.db")])
155-    assert res.exit_code == 1 and "db encrypt" in res.output
```

8. user (2026-06-10T16:22:46.524Z)

```
6-``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7-"""
8-
9:from __future__ import annotations
10-
11:import os
12:import sqlite3
13:from collections.abc import Iterator
14:from contextlib import contextmanager
15:from pathlib import Path
16-
17:from muneem.errors import EncryptedDatabaseError  # re-exported so db.EncryptedDatabaseError
works
18-
19-SCHEMA_VERSION = 4
20-
```

9. user (2026-06-10T16:22:47.432Z)

```
src\muneem\ingestion\imap.py:66:            "Attachment download and header normalization are
pending."
src\muneem\ingestion\imap.py:72:            "Attachment download and header normalization are
pending."
src\muneem\ingestion\imap.py:78:            "Attachment download and header normalization are
pending."
src\muneem\parsers\validation.py:83:    prev: float | None = None
src\muneem\parsers\validation.py:87:        if prev is not None:
src\muneem\parsers\validation.py:88:            expected = prev - (debit or 0.0) + (credit or
0.0)
src\muneem\parsers\validation.py:94:        prev = balance
src\muneem\parsers\profiles.py:4:concept?label aliases, the accepted date shape, a positional
fallback when there's no header,
src\muneem\parsers\engine.py:165:    accepted date shape, a positional fallback when there's no
header, table anchors, account-number
src\muneem\parsers\savings.py:149:    prev = opening
src\muneem\parsers\savings.py:157:            if bal < prev - tol:
src\muneem\parsers\savings.py:159:            elif bal > prev + tol:
src\muneem\parsers\savings.py:172:        prev = bal
src\muneem\parsers\savings.py:180:    and re-checked), then a coordinate-clustering fallback.
Returns ``verified=False`` if neither
src\muneem\parsers\savings.py:214:    fallback = confirmed or (clustered.rows if clustered else
[])
src\muneem\parsers\savings.py:215:    return SavingsParse(account_number, fallback, False)
```

10. user (2026-06-10T16:22:53.867Z)

```
1       """muneem's exception hierarchy.
2
3       Every error muneem raises derives from :class:`MuneemError`, so callers can catch the
whole family
4       with one `except`. Two rules for messages on these exceptions, because they may end up
```

```
in logs:
5
6       1. **Only non-sensitive context.** A file *name* is acceptable (it's the user's own
machine), but
7          never raw statement text, passwords, balances, or account numbers. Use
8          :mod:`muneem.redaction` helpers to sanitise anything derived from a document.
9       2. **Chain the cause.** Raise with ``raise ParseError(...) from exc`` so the original
traceback is
10         available for local debugging while the muneem-level message stays clean.
11      """
12
13      from __future__ import annotations
14
15
16      class MuneemError(Exception):
17          """Base class for all errors raised by muneem."""
18
19
20      class ParseError(MuneemError):
21          """A document could not be parsed (corrupt PDF, unexpected layout, extraction
failure).
22
23          Carries only non-sensitive context (e.g. the file name and the failing stage) ?
never the
24          statement contents or the underlying library's message verbatim.
25          """
26
27
28      class UnlockError(MuneemError):
29          """A password-protected document could not be unlocked (no working password, bad
input)."""
30
31
32      class EncryptedDatabaseError(MuneemError):
33          """The encrypted ledger could not be opened ? wrong key, missing SQLCipher binding,
or a
34          plaintext file where an encrypted one was expected."""
35
36
37      class KeychainError(MuneemError):
38          """Storing a secret in the OS keychain failed (no backend, locked keychain, etc.).
39
40          The secret *name* is safe to include; never the value."""
41
42
43      class ConfigError(MuneemError):
44          """A user config file (e.g. password_rules.yaml) is malformed.
45
46          Messages must reference the file by name only ? never echo its contents, which may
hold
47          password material."""
48
49
50      class MigrationError(MuneemError):
51          """A schema migration could not be applied, or the on-disk schema diverged from the
52          migration history (e.g. the frozen baseline was edited without adding a
migration)."""
53
```

11. user (2026-06-10T16:22:55.284Z)

```
34-  so the raw-hex form is used everywhere (mixing raw-key and passphrase forms silently looks
like
35-  corruption).
36-
```

```
37:**Key rotation** is implemented: `muneem db rotate-key` mints a fresh 256-bit key, re-
encrypts the
38:file (`PRAGMA rekey`), and updates the keychain ? keychain-first with rollback, so a failed
file
39:rekey leaves the keychain matching the file. Rotate on suspected compromise; no scheduled
rotation
40-needed for a single-user local key.
41-
42:### Disaster recovery ? the key-loss problem (solved)
43-
44-Because the key lives only in the keychain and `db backup` encrypts backups with that *same*
key, a
45-**lost keychain** (OS reinstall, new machine, profile corruption) would otherwise destroy the
ledger
46:*and every backup*. So a **passphrase-wrapped recovery file** is implemented
(`muneem.recovery`):
47-
48:- **`muneem db setup-recovery`** ? wraps the current DB key under a user passphrase
49:  (**Argon2id** KDF ? **AES-256-GCM**) into `muneem-recovery.json`. The file holds only the
*wrapped*
50-  key (salt/nonce/ciphertext + KDF params) ? never the key, never the passphrase. **Store it
51:  off-machine** (password manager / separate drive). The pre-commit guard blocks
`*recovery*.json`
52-  from the repo.
53:- **`muneem db recover <file>`** ? prompts for the passphrase, unwraps the key (a wrong
passphrase
54-  fails the GCM tag ? clean error, no oracle), verifies it opens the ledger, and restores it
to the
--
56-
57-**Deferred (opt-in, ROADMAP D.2 Phase 2):** a full passphrase-*lock* mode (the working key
itself
58-derived from a passphrase, never stored) for stronger-than-keychain protection on a shared
machine ?
59:distinct from the recovery file above, which keeps the frictionless keychain key as primary.
60-
61-## Threat model ? stated honestly
62-
```

12. user (2026-06-10T16:23:19.797Z)

```
1       """Secret storage via the OS keychain.
2
3       OAuth tokens, cached statement passwords, and the DB encryption key are stored here
(Windows
4       Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5       in logs. See DESIGN.md section 5.
6
7       External-call discipline (every keyring call is wrapped with attribution):
8       - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
``None``
9         (absence is a valid state; this also keeps headless CI happy).
10      - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11        because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12        included for diagnosis; the value never is.
13      """
14
15      from __future__ import annotations
16
17      import keyring
18      from keyring.errors import KeyringError, PasswordDeleteError
19
```

```
20      from muneem.errors import KeychainError
21
22      _SERVICE = "muneem"
23
24
25      def get_secret(name: str) -> str | None:
26          """Return the stored secret, or ``None`` if absent or the keychain is
unavailable."""
27          try:
28              return keyring.get_password(_SERVICE, name)
29          except KeyringError:
30              return None
31
32
33      def set_secret(name: str, value: str) -> None:
34          """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35          try:
36              keyring.set_password(_SERVICE, name, value)
37          except KeyringError as exc:
38              # Name only ? never the value ? in the message (it may be a password or the DB
key).
39              raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42      def delete_secret(name: str) -> None:
43          """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44          try:
45              keyring.delete_password(_SERVICE, name)
46          except (PasswordDeleteError, KeyringError):
47              pass
48
```

13. user (2026-06-10T16:24:11.974Z)

Structured output provided successfully

14. assistant (2026-06-10T16:24:15.516Z)

Verification complete.